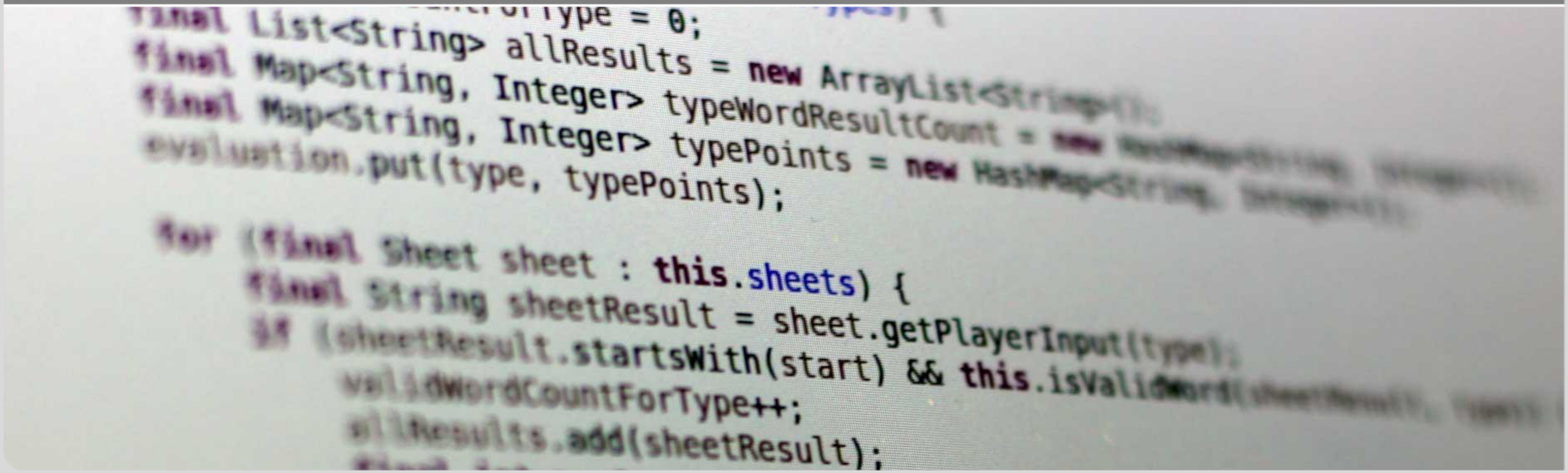
Karlsruher Institut für Technologie

DHBW
Duale Hochschule
Baden-Württemberg

Programmieren I

Entwicklungsumgebungen Eclipse und IntelliJ IDEA

Institut für Automation und angewandte Informatik



Was ist Programmieren

- Quellcode schreiben

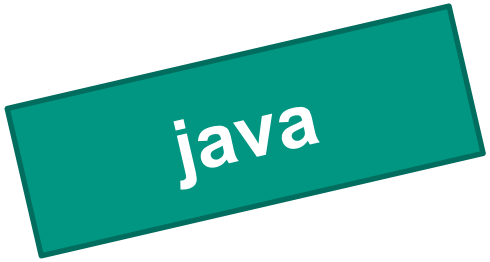


- Der Maschine verständlich machen („kompilieren“)



- Mit Standardbibliotheken verknüpfen („linken“) // nicht in Java

- Erzeugtes Programm ausführen und Fehler beheben („Debuggen“)



Vereinfachung: Ein Tool für alles!

- Entwicklungsumgebungen bieten typischerweise Werkzeuge zum
  - Quellcode editieren
  - Code übersetzen
  - Linken
  - Ausführen
  - Debuggen

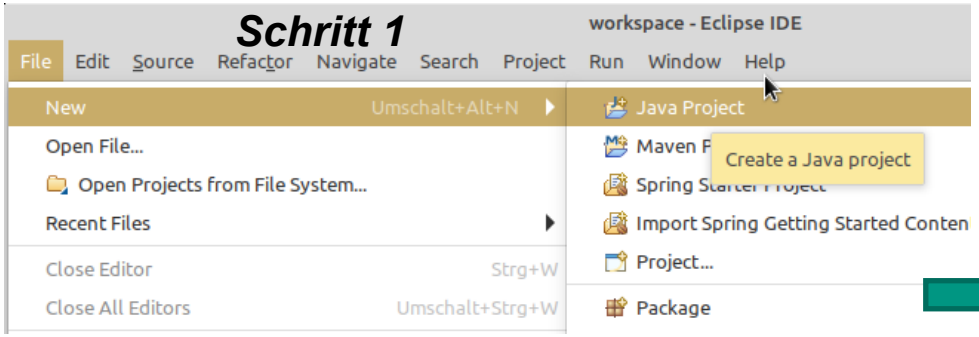
- Auch meist vorhanden:
  - Syntax-Highlighting
  - „Live“-Anzeige von Syntax-Fehlern
  - Unterstützung in Form verschiedener Code-Generatoren
  - Integration in verschiedene Tools (z.B. Versionskontrollsysteme)

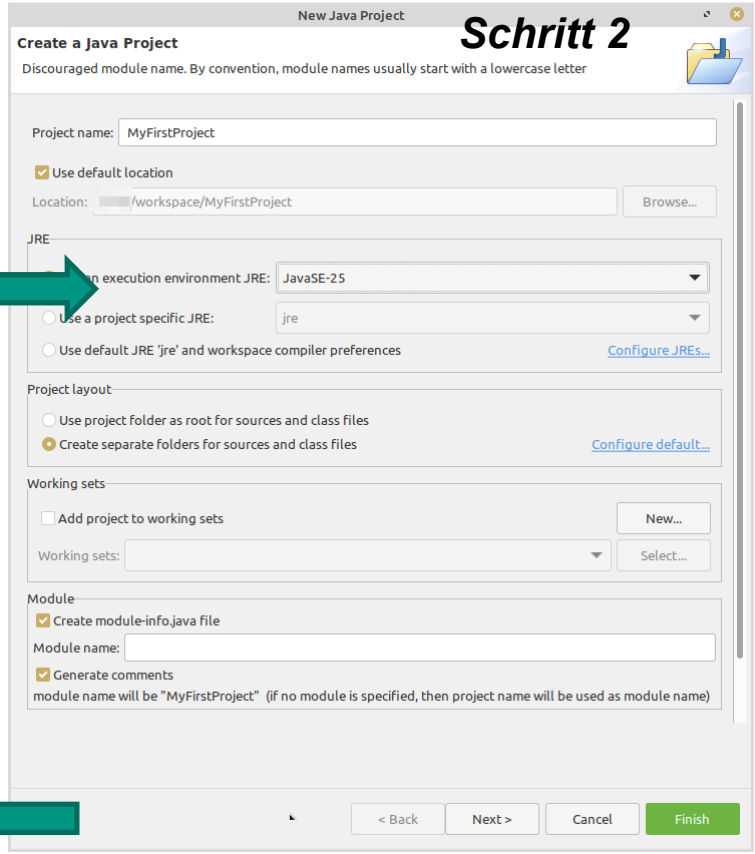
Entwicklungsumgebungen für Java

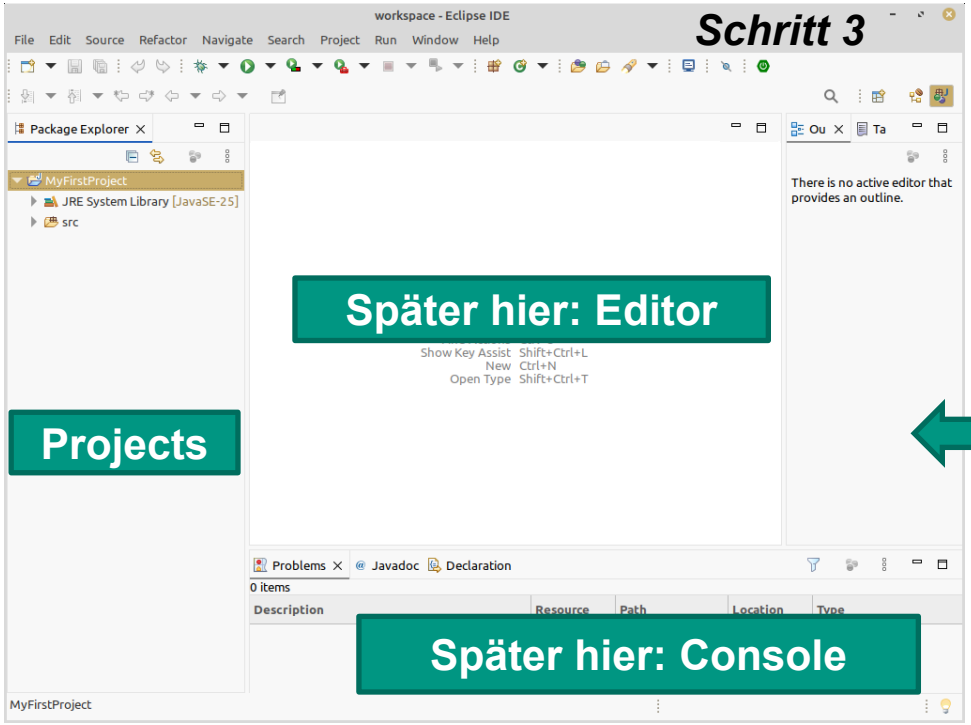
- Java Development Kit (JDK)
  - openJDK, bspw. Eclipse Temurin: https://adoptium.net/de/temurin
- Englisch: *Integrated Development Environment* → IDE
- Eclipse (aktuell: Version 2025-12 / 4.38)
  http://www.eclipse.org/
- IntelliJ IDEA (aktuell: Version 2025.3.1)
  https://www.jetbrains.com/idea/

Stand: 05. Januar 2026

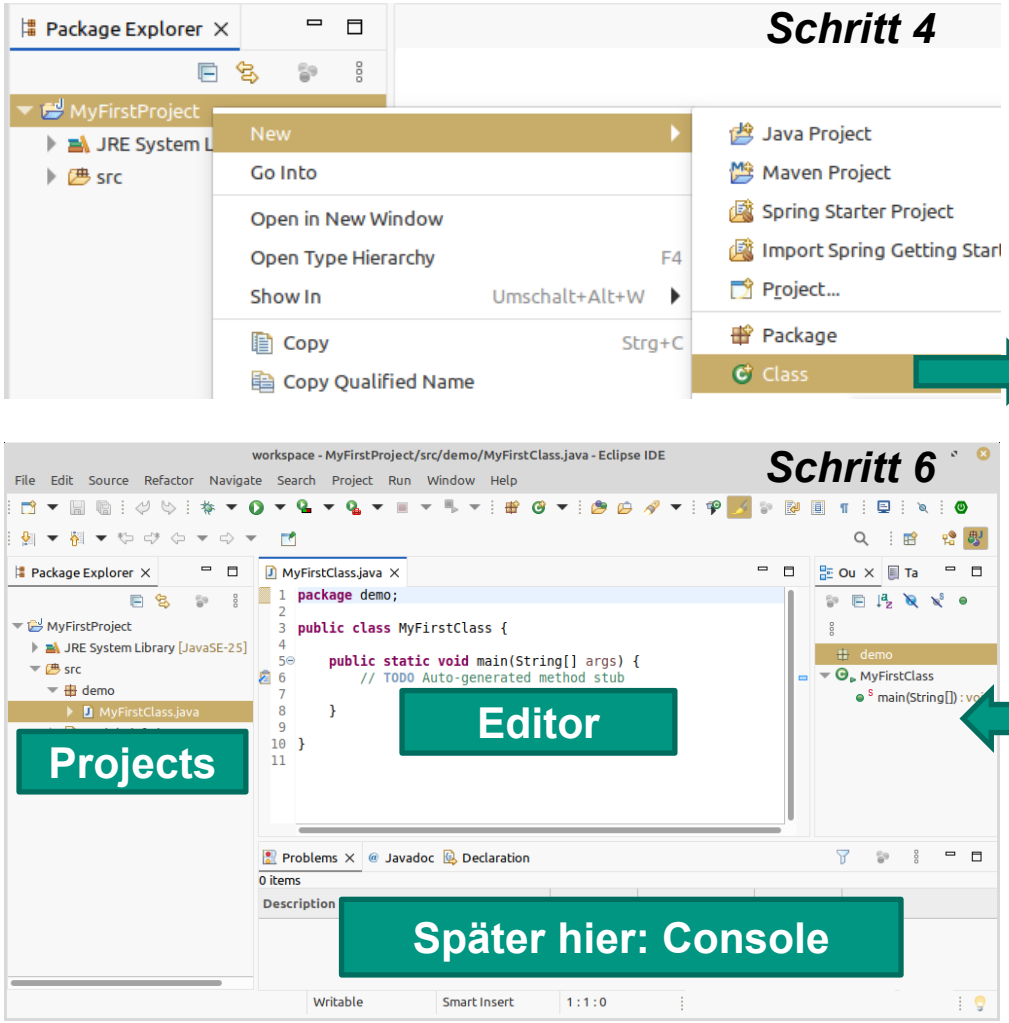
Neues Projekt (1) – Eclipse I

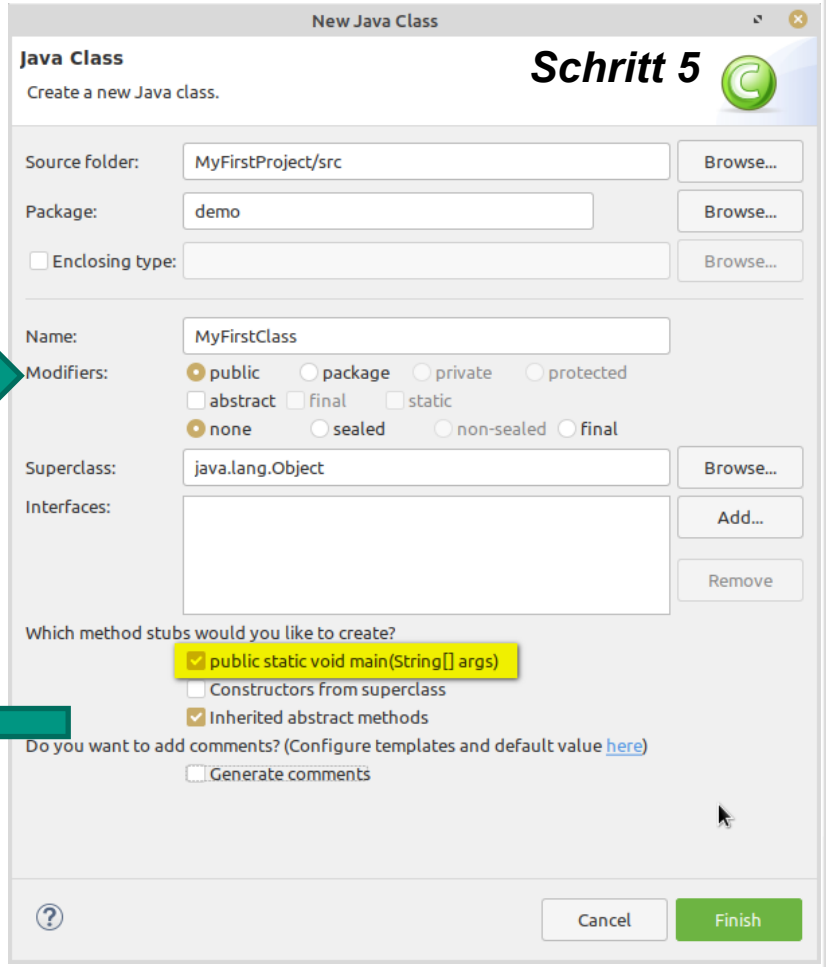






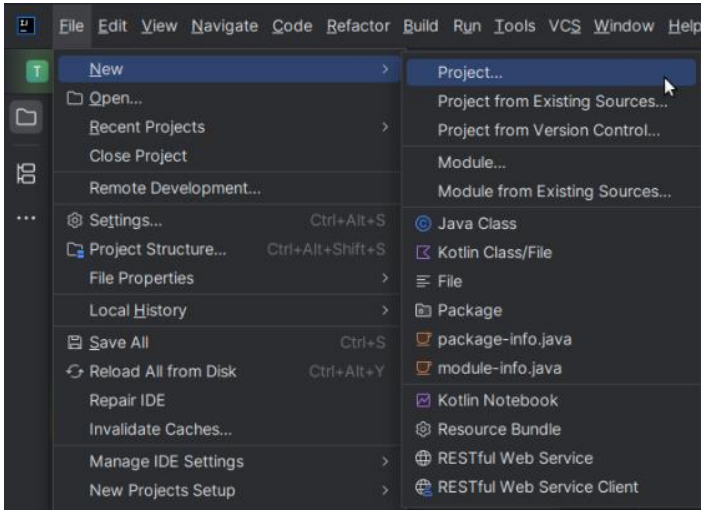
Neues Projekt (2) – Eclipse II



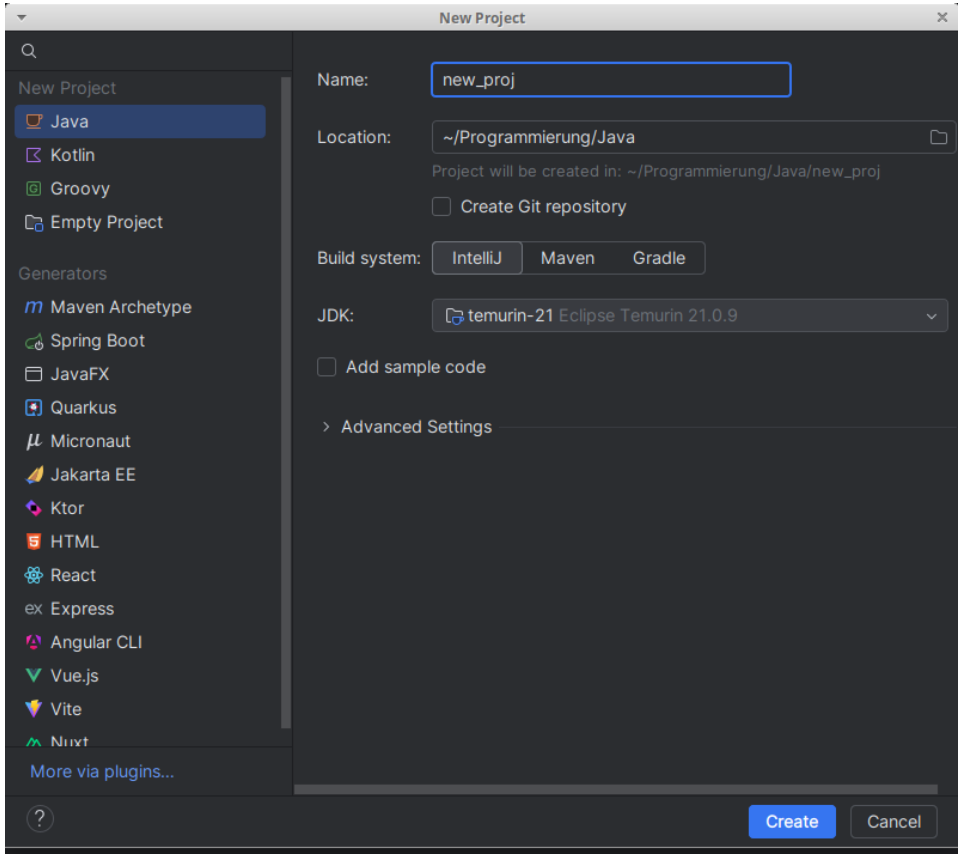


Neues Projekt (3) – IntelliJ IDEA I

Schritt 1



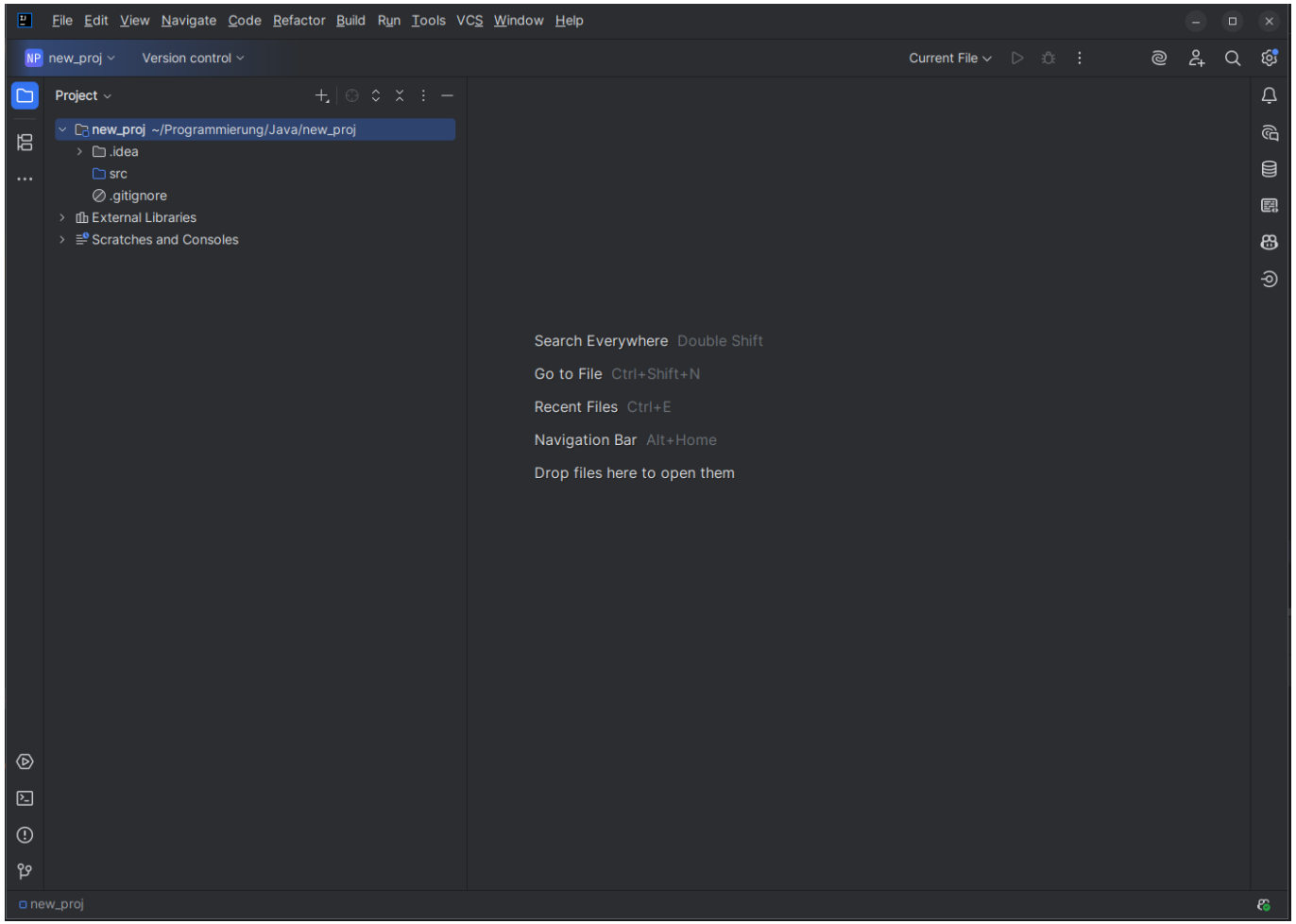
Schritt 2



Create

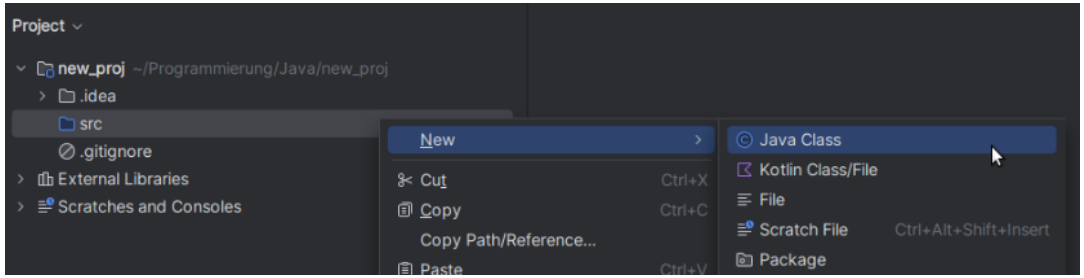
Neues Projekt (4) – IntelliJ IDEA II

Schritt 3: Projekt bereits startklar!

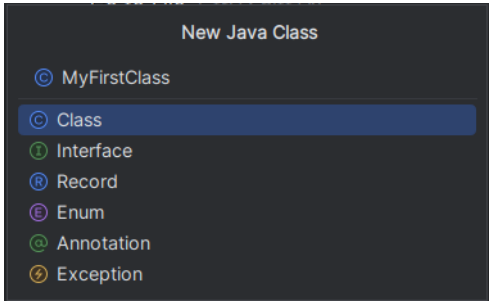


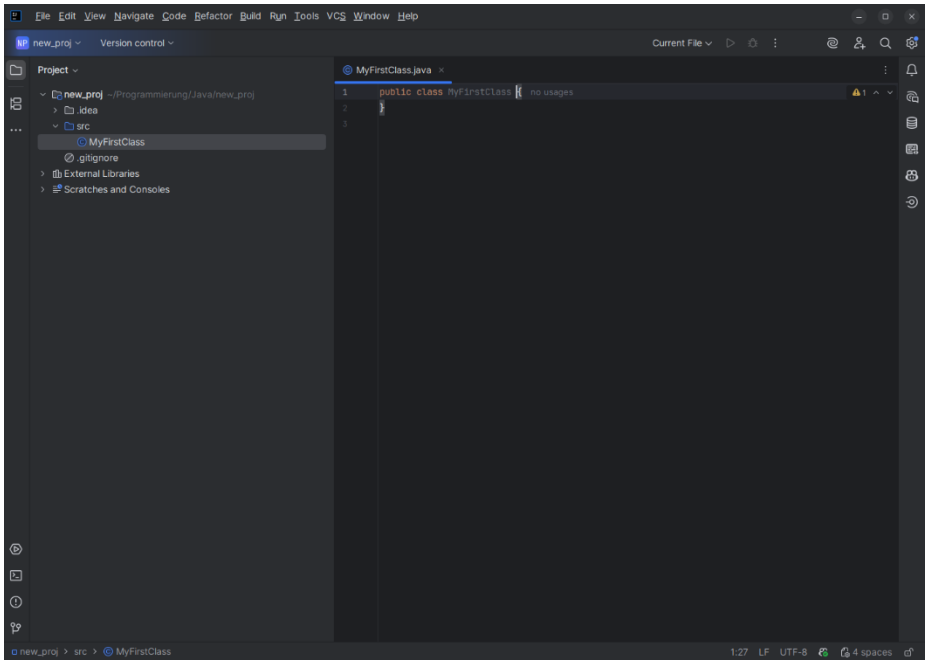
Neues Projekt (5) – IntelliJ IDEA III

Schritt 4



Schritt 5

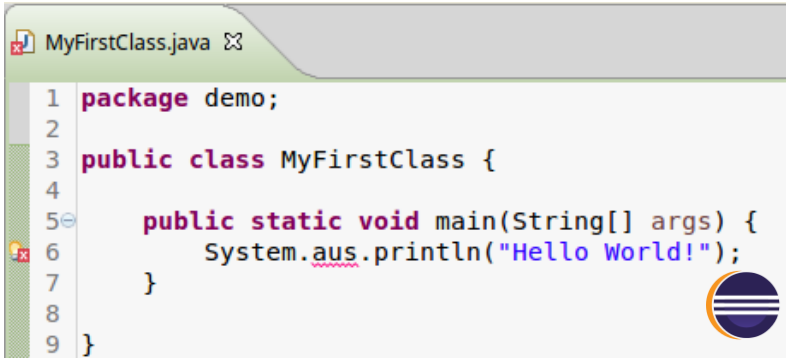


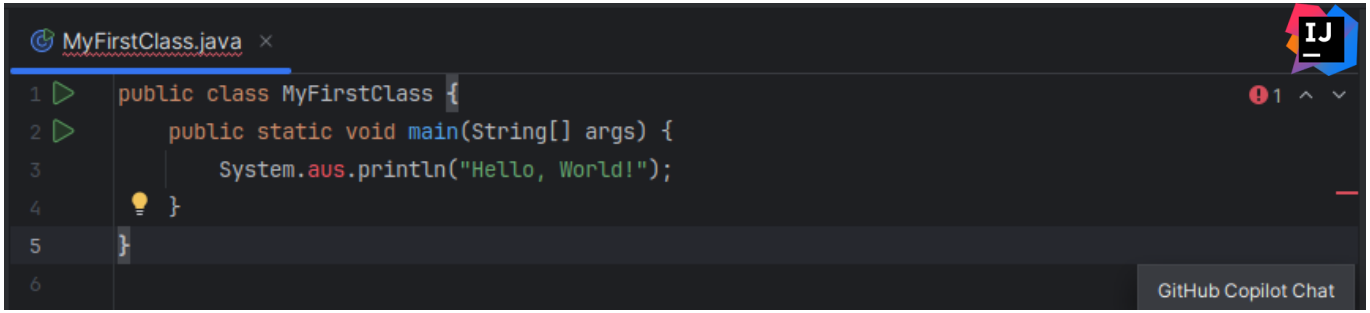


Schritt 6: Ready to start coding!

Projekt compilieren

- IDEs übersetzen Java-Dateien beim Speichern in der Regel automatisch und zeigen ggf. Fehler an

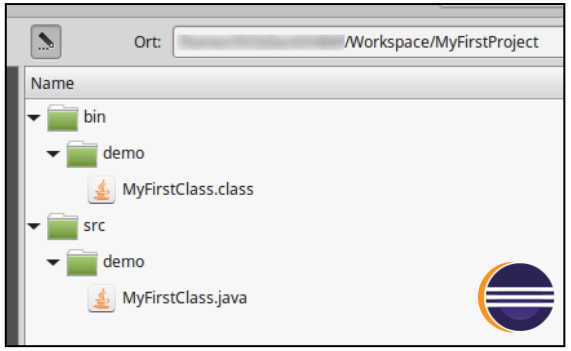


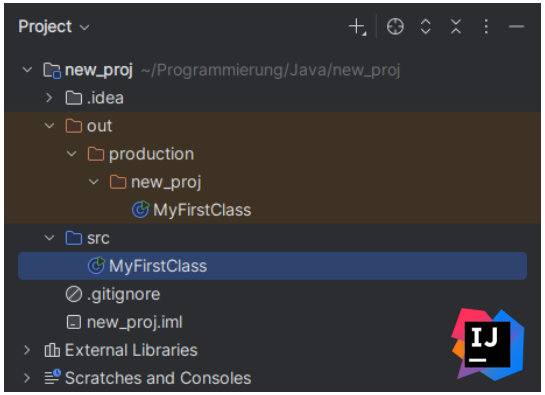


Dateien und Dateibaum im Projektverzeichnis

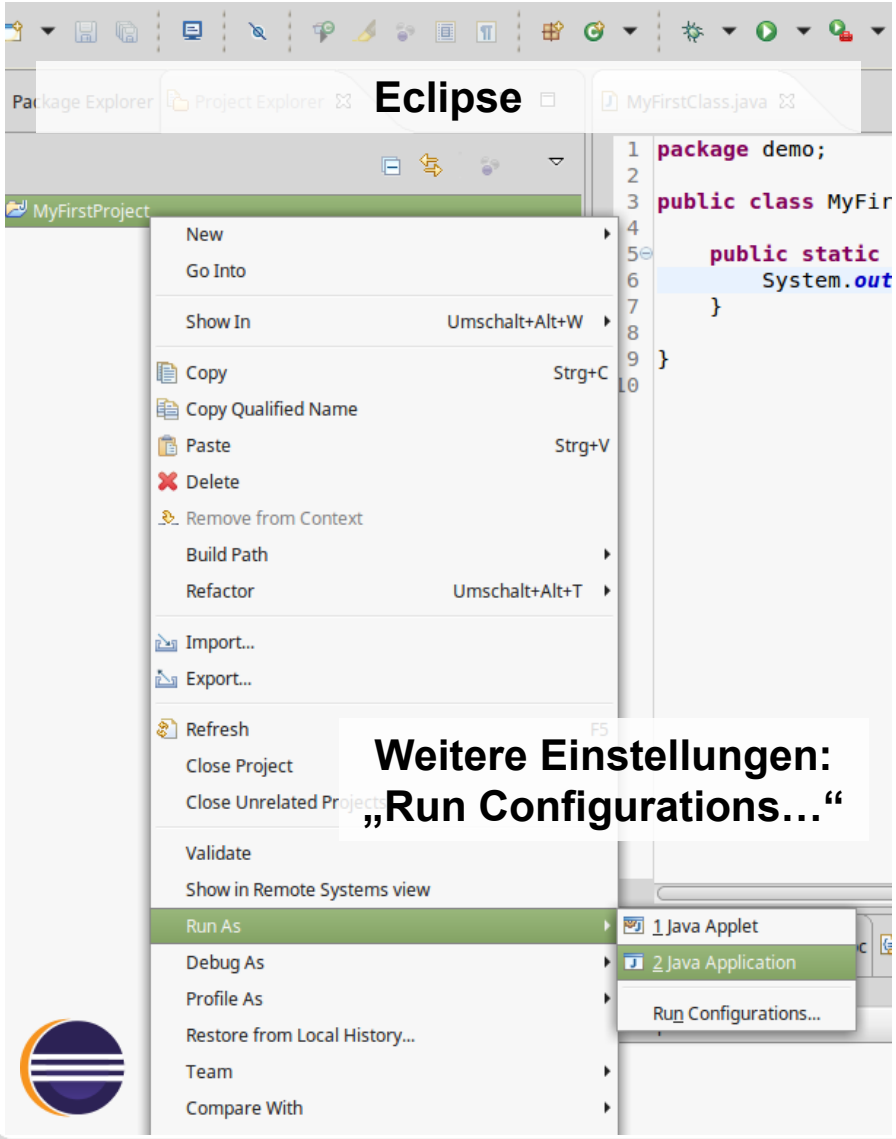
- **`src`** („Source“)
  → Quellcode / `.java`-Dateien

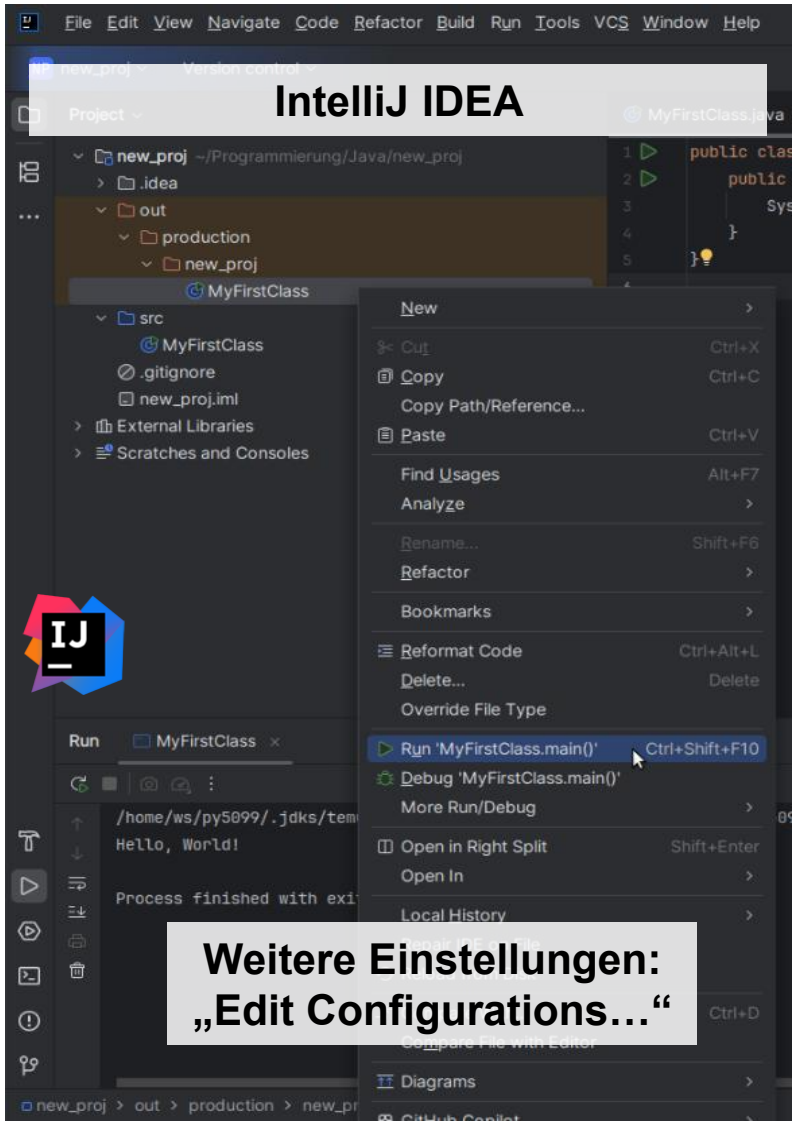
- **`bin`** (Eclipse)
  `out` (IntelliJ IDEA)
  → Bytecode / `.class`-Dateien





Programm ausführen





Effektives Arbeiten: Tastaturkürzel / Hotkeys

- Viele Aktionen können durch geschickten Einsatz von Tastatur oder Maus vereinfacht werden
  → Beschleunigung des Arbeitsablaufs

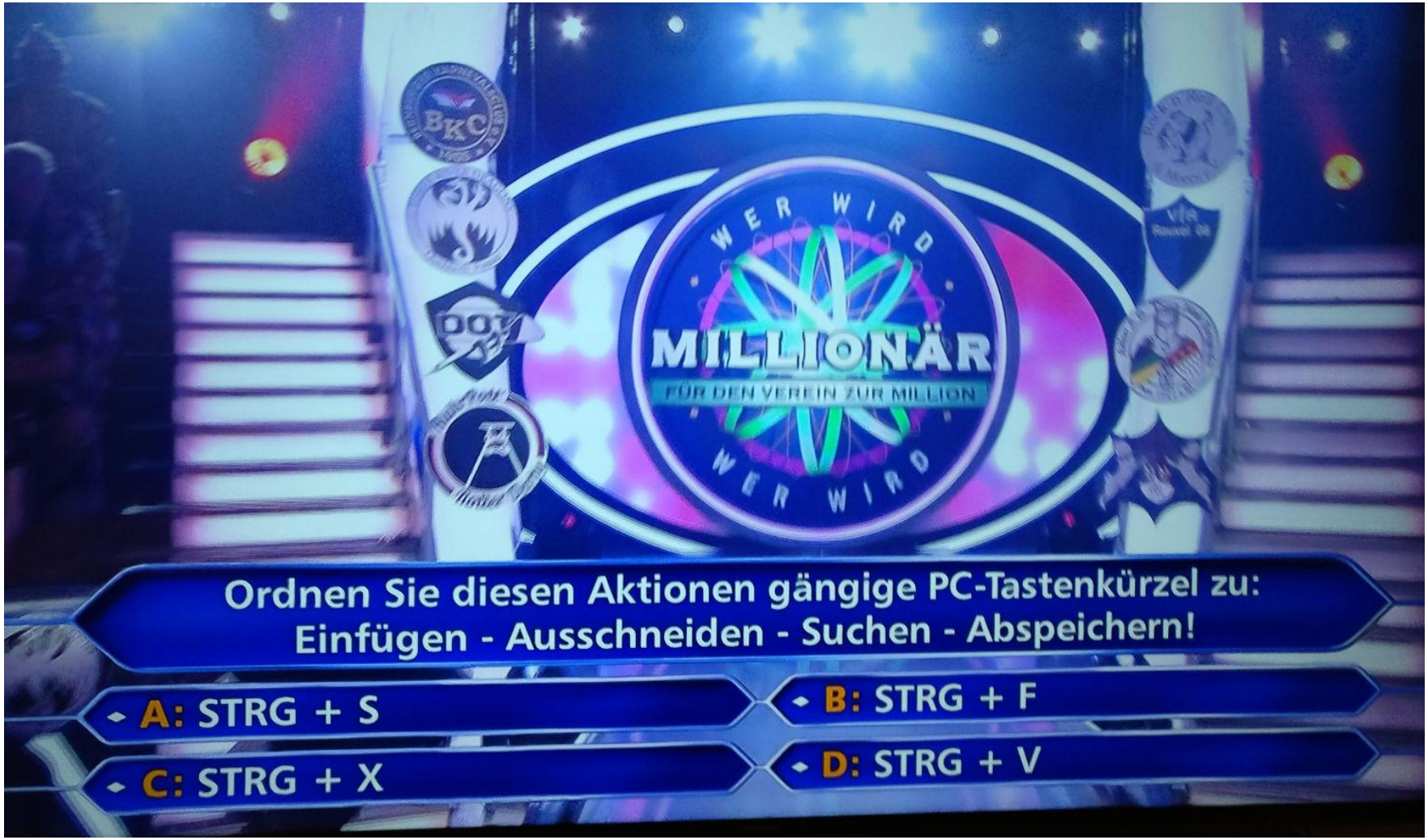
- Viele Tastatur- und Mausaktionen sind programmübergreifend gleich

- Viele Werkzeuge halten Standards zu Tastaturbelegungen bereit

Systemweite Tastenkombinationen

- **Strg** + **C** Selektierten Text kopieren (in Zwischenablage)
- **Strg** + **X** Selektierten Text ausschneiden (in Zwischenablage)
- **Strg** + **V** Inhalt der Zwischenablage einfügen
- **Strg** + **P** Drucken
- **Strg** + **S** Speichern
- **Strg** + **Z** Aktion rückgängig machen („Undo“)
- **Strg** + **Y** Aktion erneut ausführen („Redo“)
- **Strg** + **A** Gesamten Text selektieren

Man kann damit sogar reich werden!



Quelle: RTL Television / „Wer wird Millionär? Für den Verein zur Million“ (11.12.2017)

Positionierung des Cursors mit der Tastatur

- **Frage:** Wozu? Dafür gibt es ja die Maus!
- **Antwort:** Meist viel schneller, da Texteingabe sowieso mit Tastatur
- Den Cursor …
  - … um einzelne Zeichen und Zeilen versetzen: → ← ↑ ↓
  - … um einzelne Worte versetzen: Strg + → / ←
  - … an Anfang/Ende der Zeile versetzen: Pos1 Ende
  - … an Anfang/Ende der Datei versetzen: Strg + Pos1 / Ende
  - … an zuletzt editierte Position springen: Strg + Q 
    Shift + F5

Selektieren von Text mit der Tastatur

- **Frage:** Wozu? Dafür gibt es ja die Maus!
- **Antwort:** Innerhalb eines Absatzes meist schneller
- Selektion (von der aktuellen Cursorposition aus) …
  - … einzelner Zeichen: **Shift** + → / ←
  - … einzelner Zeilen: **Shift** + ↑ / ↓
  - ... einzelner Worte: **Strg** + **Shift** + → / ←
  - … bis Anfang/Ende der Zeile: **Shift** + **Pos1** / **Ende**
  - … bis Anfang/Ende der Datei: **Strg** + **Shift** + **Pos1** / **Ende**

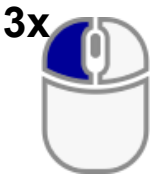
Positionierung des Cursors mit der Maus

- Vorteil der Maus: „große Strecken“ zurücklegen
- Selektion (ohne Rücksicht auf aktuelle Cursorposition) …
  - … einzelner Worte (zwischen 2 Trennzeichen, z.B Leerzeichen, Punkt oder Klammer)
  - … einzelner Zeilen
  - … Selektion erweitern

Doppelklick



Dreifach-Klick

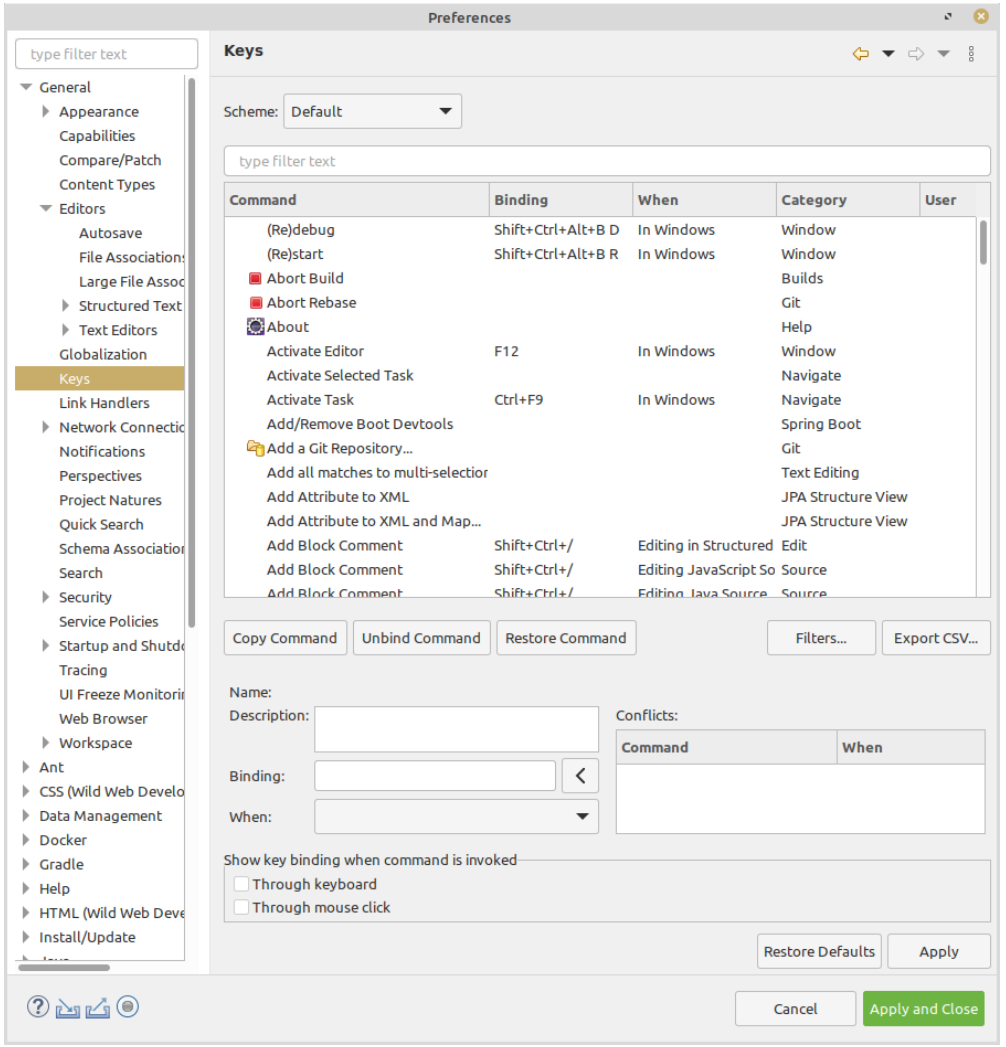


gedrückt halten und Maus ziehen



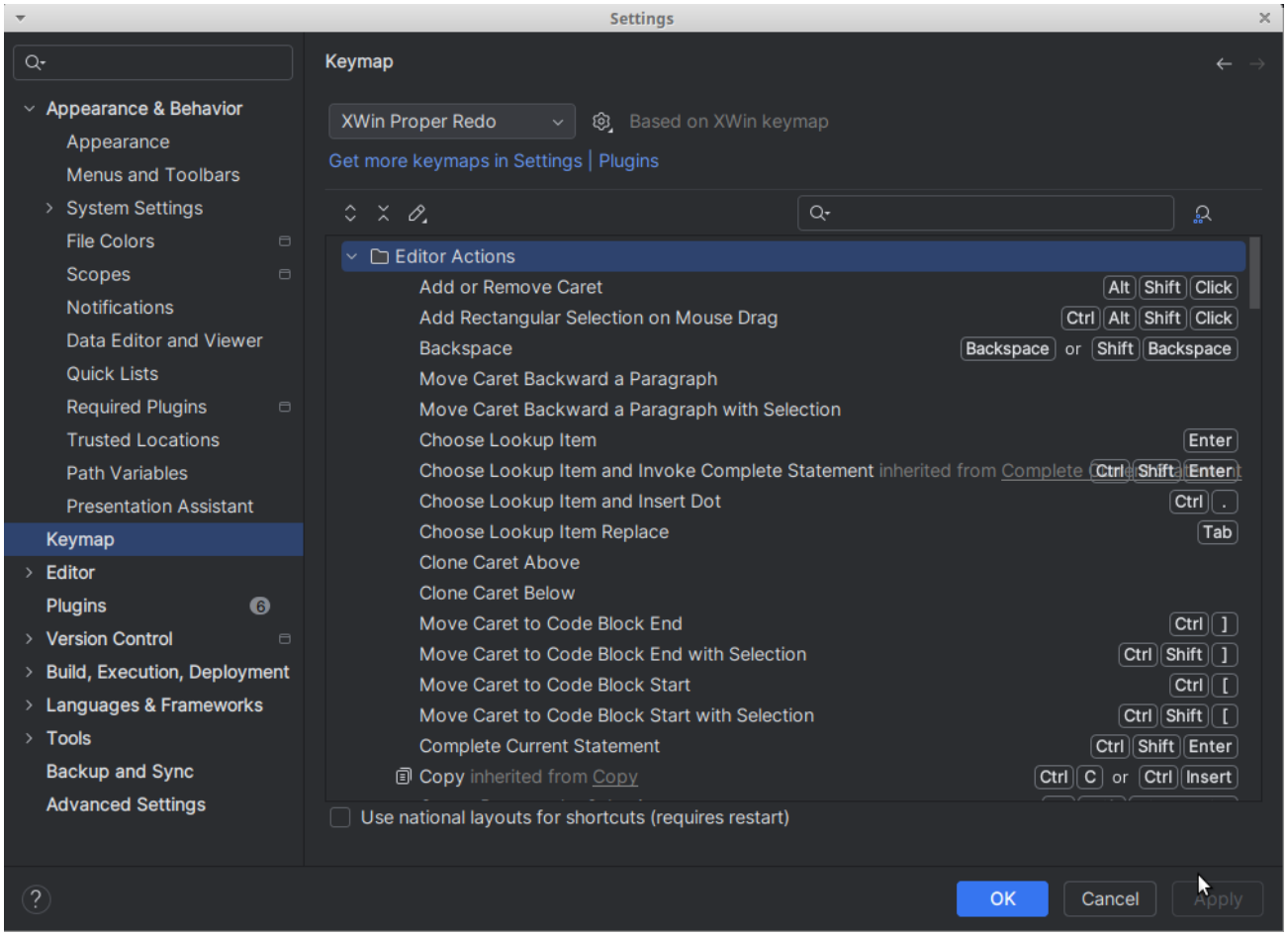
Tastenkombinationen in Eclipse

- Window
  - ↓ Preferences
  - ↓ General
  - ↓ Editors
  - ↓ Keys



Tastenkombinationen in IntelliJ IDEA

- File
  ↓
  Settings
  ↓
  Keymap



Nützliche Beispiele (Linux + Windows)

Aktion	Eclipse	IntelliJ IDEA
Programm ausführen	STRG + F11	SHIFT + F10
Aktuelle Zeile löschen	STRG + D	STRG + Y
Aktuelle Zeile kopieren	STRG+ALT + ↑ / ↓	STRG + D
In aktueller Datei suchen	STRG + F	STRG + F
Im Projekt suchen	STRG + H	STRG + SHIFT + F
Parameter-Info	STRG+SPACE (am Anfang der Argumente)	STRG + P

Nützliche Beispiele (MacOS)

Aktion	Eclipse	IntelliJ IDEA
Programm ausführen	Cmd + Shift + F11	Ctrl + Option + R
Aktuelle Zeile löschen	Cmd + D	Cmd + Backspace
Aktuelle Zeile kopieren	Option + Cmd + ↑ / ↓	Cmd + D
In aktueller Datei suchen	Cmd + F	Cmd + F
Im Projekt suchen	Cmd + H	Cmd + Shift + F
Parameter-Info	Cmd + Space (am Anfang der Argumente)	Cmd + P

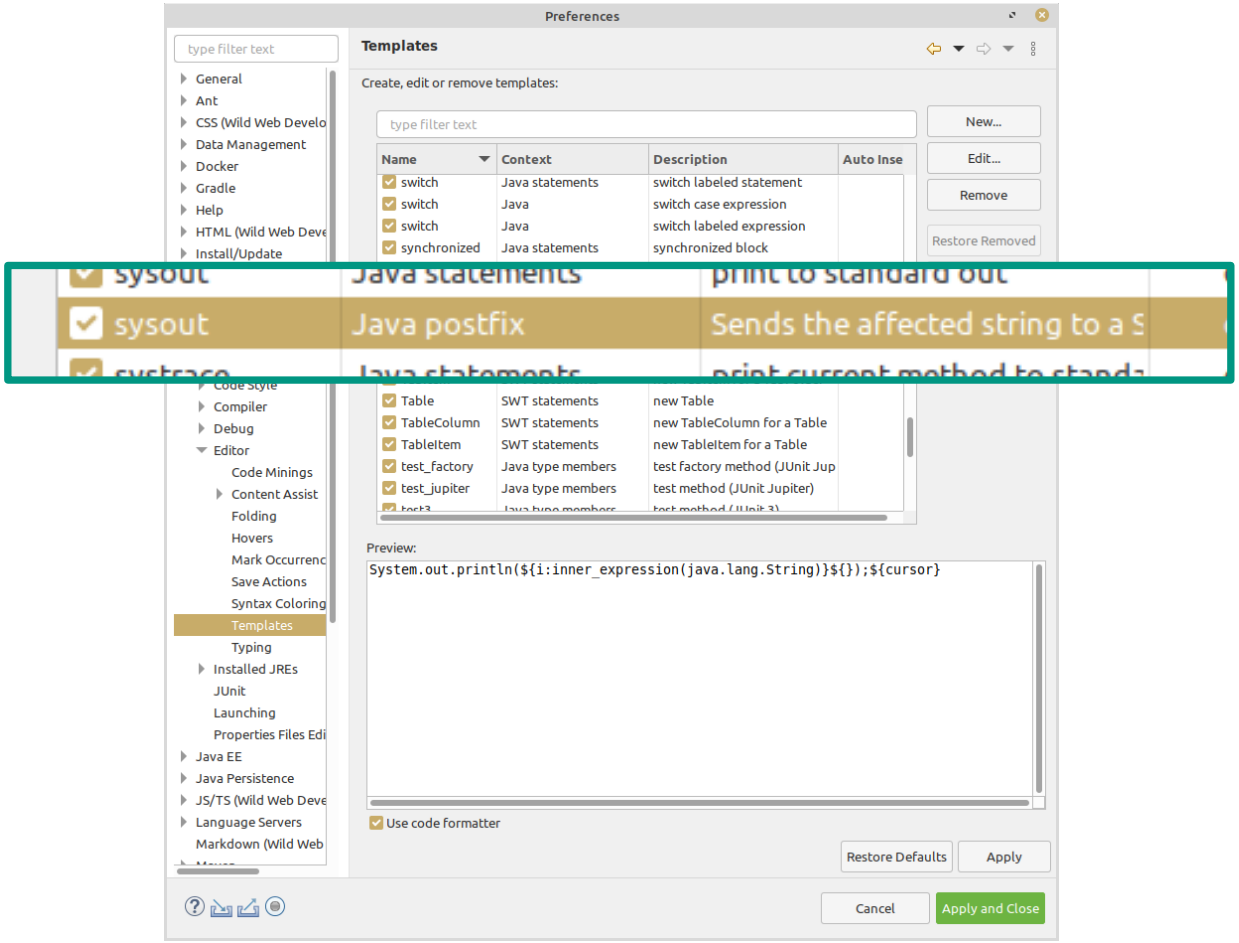
Code-Templates

- Code-Templates sind vorgefertigte Schablonen für längere (und häufig genutzte) Quellcode-Fragmente, die aus Abkürzungen erzeugt werden können
- Gerade zu Beginn häufiges Beispiel: Ausgabe auf Konsole
  - Statt `System.out.println("");` nur `sysout` bzw. `sout` tippen
- Abkürzungen und Anwendung können sich je nach IDE unterscheiden

Code-Templates - Eclipse

- Anwendung: **Strg** + **Space**

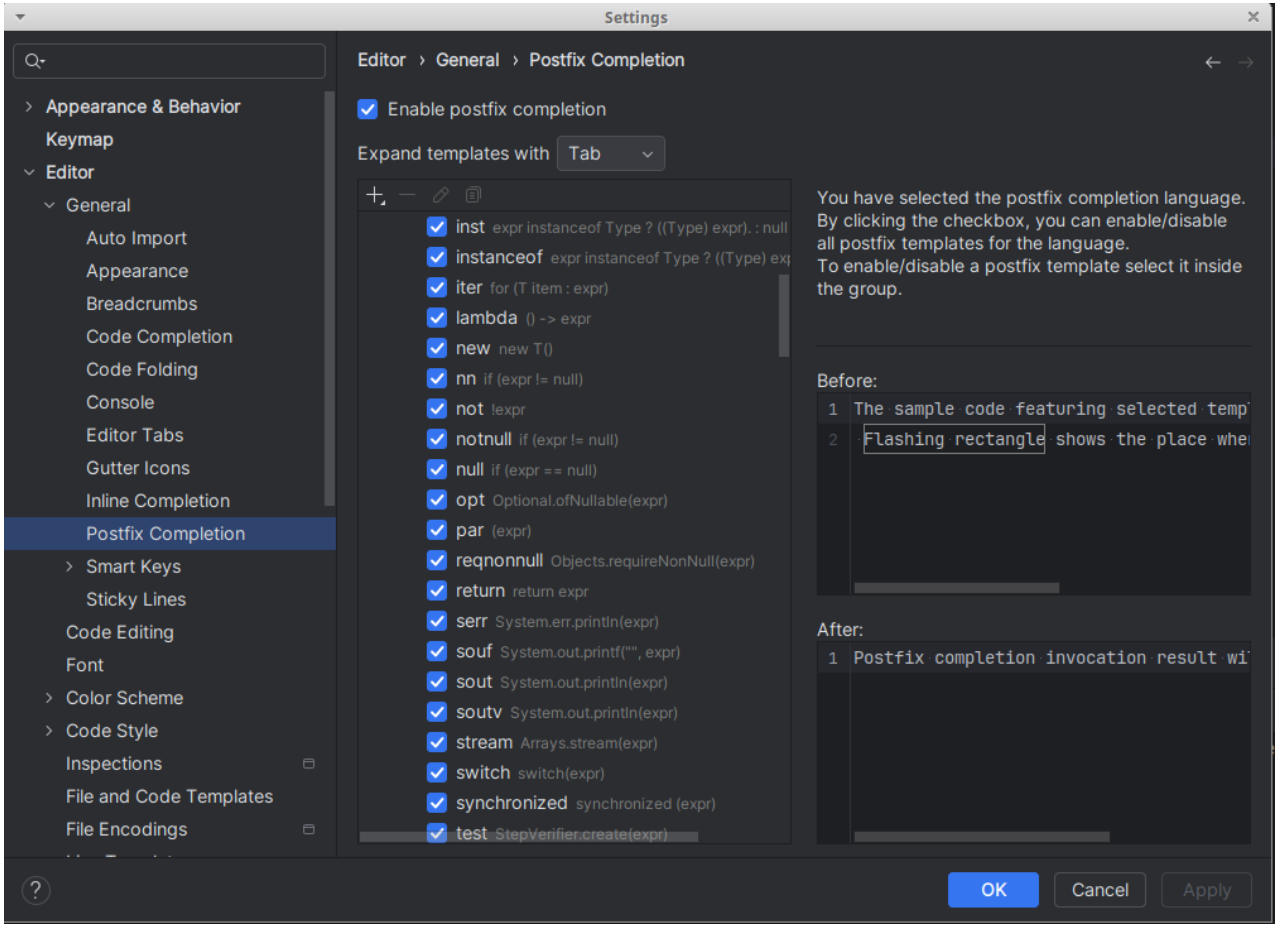
- Window
  - ↓
  - Preferences
  - ↓
  - Java
  - ↓
  - Editor
  - ↓
  - Templates



Code-Templates - IntelliJ IDEA

- Anwendung: meist einfach Tippen

- File
  ↓
  Settings
  ↓
  Editor
  ↓
  General
  ↓
  Postfix Completion



Weitere Hilfen

- **Wichtig:**
  Werkzeug zum täglichen Arbeiten sollte man gut kennen!

- Lästige „Standard-Arbeiten“ können einem moderne Entwicklungsumgebungen oft abnehmen

- Üben, Üben, Üben …

➔ Resultat:
große Zeitersparnis und effektiveres Programmieren

selbst ausprobieren

Programmierunterstützung per KI (1)

- Seit einigen Jahren können KI-basierte Tools beim Programmieren unterstützen, bieten z.B. eine erweiterte Code-Completion bis hin zur Erzeugung von ganzen Klassen(-strukturen), Methoden, etc.

- Beispiele:
  - GitHub Copilot / Microsoft 365 Copilot
  - Claude.ai
  - Google Gemini
  - Tabnine
  - Figstack
  - SourceGraph
  - Mutable.ai
  - …



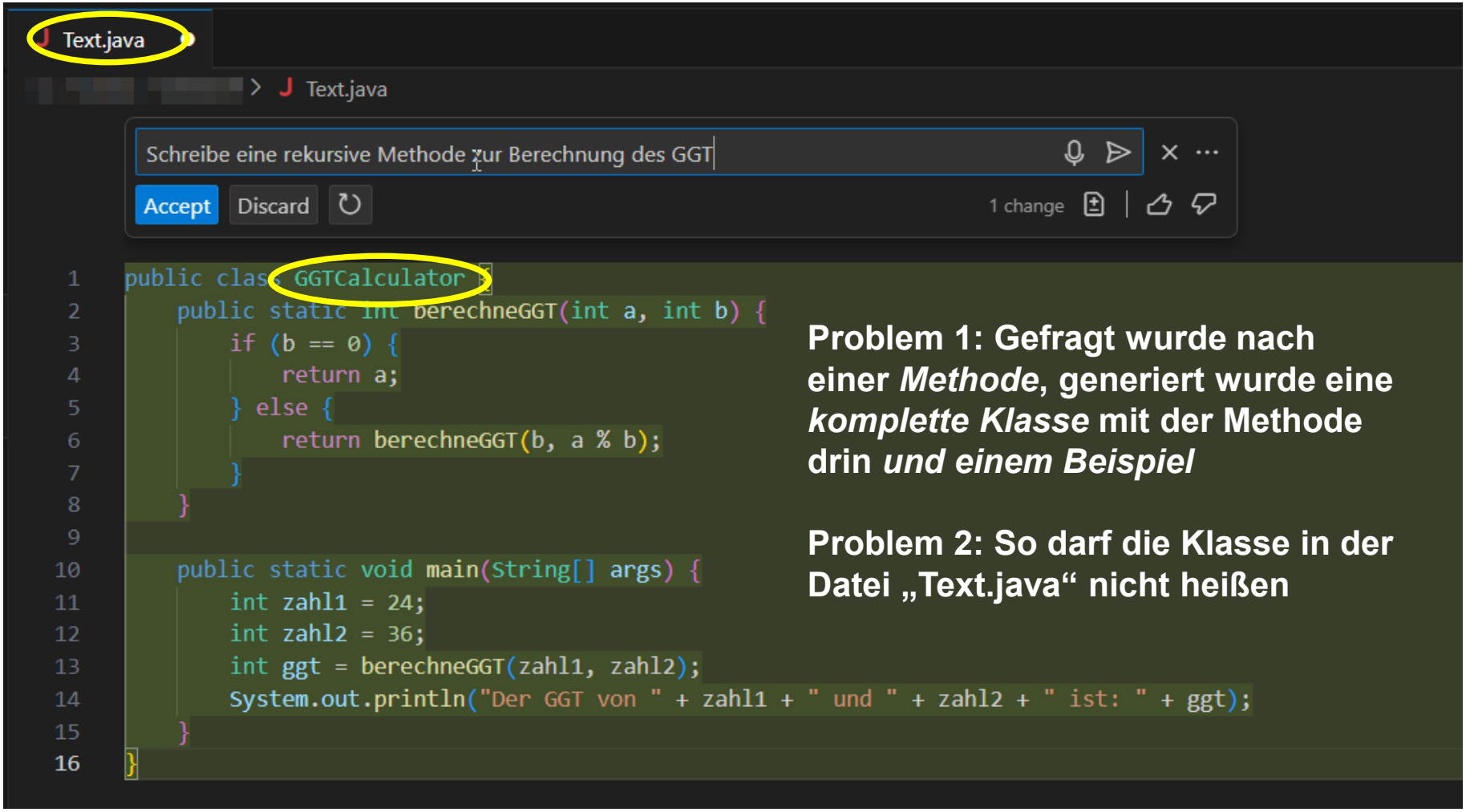
Programmierunterstützung per KI (2)

- Die Verfügbarkeit dieser Tools hat Auswirkungen auf das Berufsbild von Programmierern/Software-Ingenieuren:
  - Entlastung von Routineaufgaben
  - Automatisierung von Standard-Prozessen
  - Kompetenz, erzeugten Code auf Eignung und Qualität hin zu überprüfen und ggf. anzupassen



Quelle: https://youtu.be/8PpEgeAbua0

Einfaches Beispiel

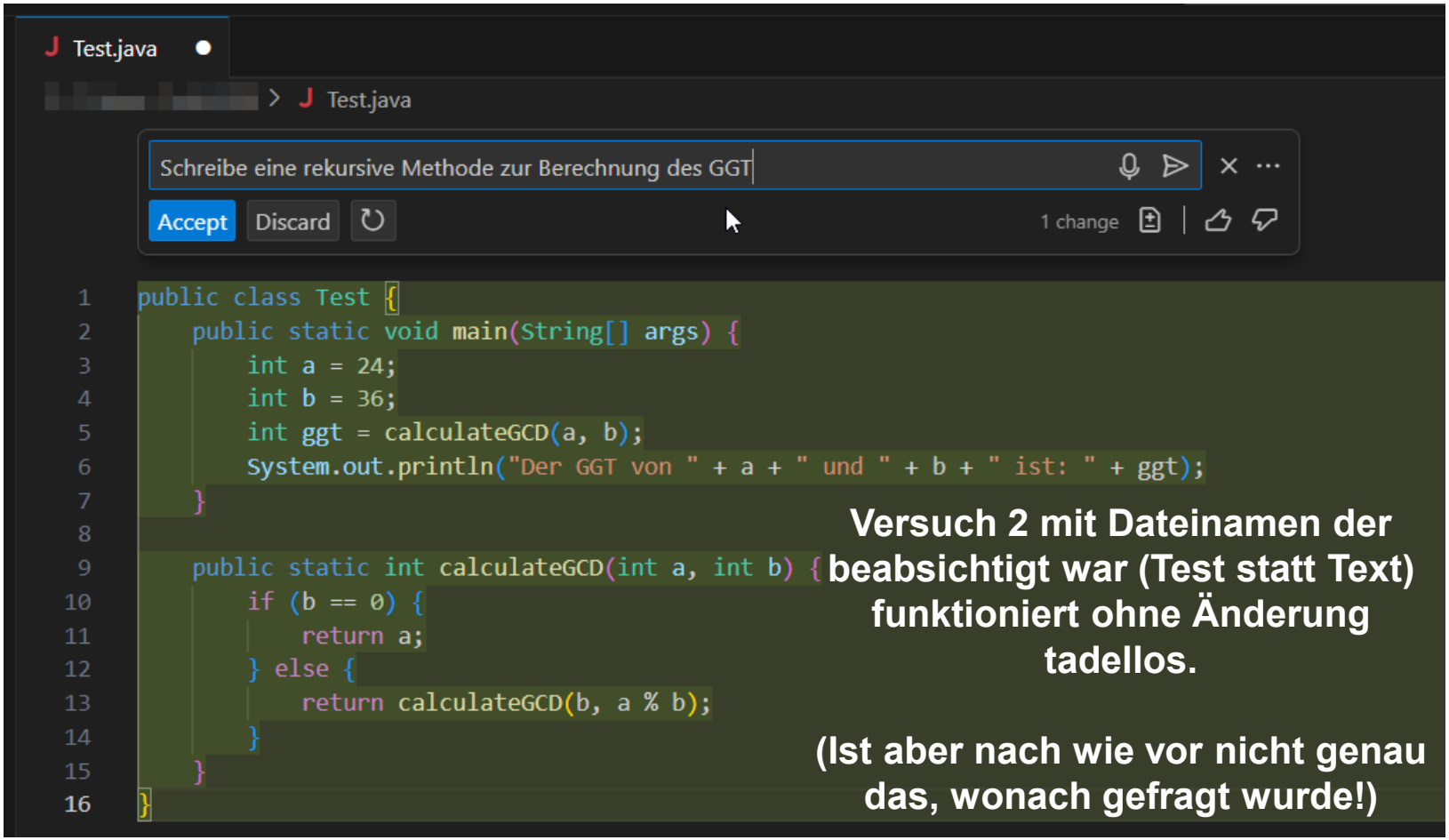


Problem 1: Gefragt wurde nach einer *Methode*, generiert wurde eine *komplette Klasse* mit der Methode drin *und einem Beispiel*

Problem 2: So darf die Klasse in der Datei „Text.java“ nicht heißen

Genutzte KI: GitHub Copilot Business, Dez. 2024

Zur Wahrheit gehört auch…

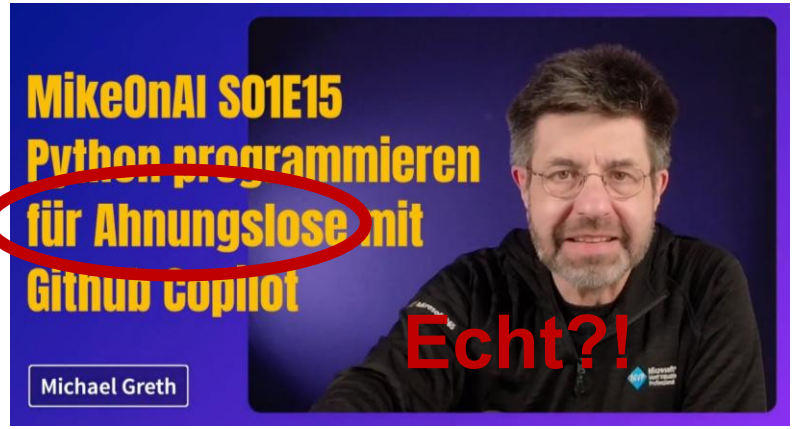


Genutzte KI: GitHub Copilot Business, Dez. 2024

„Uffbasse!“

- Das ist ein sehr trivialer Fall, zeigt aber: Ahnungslose…
  - … haben keine Ahnung, warum Beispiel 1 vom Compiler nicht akzeptiert wird, Beispiel 2 aber schon
  - … merken nicht, dass sie nicht genau das bekommen haben, wonach gefragt wurde
  - … würden ggf. Programme voller nutzloser Beispiele produzieren

- Was ist dann erst mit inhaltlichen Fehlern, die *nicht trivial* zu erkennen sind?



Quelle: https://youtu.be/8PpEgeAbua0

Noch ein drittes Beispiel

- Kompliziert zu lesen für Menschen, aber für Maschinen sehr trivial:

```
int c = 1 +-+-+-+-+ 1

kannst du mir sagen, was dies bedeutet?
```

- An diesem Beispiel wollen wir uns auch die „Evolution“ der KI-Modelle anschauen
  → daher zunächst die Antworten von **vor einem Jahr**!

Noch ein drittes Beispiel

Dezember 2024!

int c = 1 +-+-+-+-+ 1

kannst du mir sagen, was dies bedeutet?

GitHub Copilot

Der Ausdruck `int c = 1 +-+-+-+-+ 1` ist syntaktisch korrekt, aber er ist schwer lesbar und verwirrend. Die Zeichenfolge `+-+-+-+` besteht aus abwechselnden Plus- und Minuszeichen, die sich gegenseitig aufheben.

Hier ist eine detaillierte Aufschlüsselung:

1. `1 + - 1` ergibt `0`
2. `0 + - 1` ergibt `-1`
3. `-1 + - 1` ergibt `-2`
4. `-2 + - 1` ergibt `-3`
5. `-3 + - 1` ergibt `-4`

Der Ausdruck kann also vereinfacht werden zu:

```
int c = 1 - 1; // was 0 ergibt
```

Der Wert von `c` wird also `0` sein.

Genutzte KI: gpt-4o via GitHub Copilot Business, Dez. 2024

Probieren wir es doch mal aus!

```
int c = 1 +-+-+-+-+ 1
```

kannst du mir sagen, was dies bedeutet?

GitHub Copilot

Der Ausdruck `int c = 1 +-+-+-+-+ 1` ist syntaktisch korrekt, aber er ist schwer lesbar und verwirrend. Die Zeichenfolge `+-+-+-+` besteht aus abwechselnden Plus- und Minuszeichen, die sich gegenseitig aufheben.

Hier ist eine detaillierte Aufschlüsselung:

1. `1 + - 1` ergibt `0`
2. `0 + - 1` ergibt `-1`
3. `-1 + - 1` ergibt `-2`
4. `-2 + - 1` ergibt `-3`
5. `-3 + - 1` ergibt `-4`

Der Ausdruck kann also vereinfacht werden zu:

```
int c = 1 - 1; // was 0 ergibt
```

Der Wert von `c` wird also `0` sein.

```
1  package test;
2
3  public class Test {
4
5      public static void main(final String[] args) throws Exception {
6          final int c = 1 +-+-+-+-+ 1;
7          System.out.println("Ergebnis ist: " + c);
8      }
```

Search | Git Staging | Console | Problems

rogram Files\Corretto17\jdk17.0.13_11\bin\javaw.exe (20.12.

```
Ergebnis ist: 2
```

Dezember 2024!

Schade!

Genutzte KI: gpt-4o via GitHub Copilot Business, Dez. 2024

Jeder hat eine zweite Chance verdient!

```
int c = 1 - 1; // was 0 ergibt
```

Der Wert von `c` wird also `0` sein.



bist du dir sicher?

GitHub Copilot

Entschuldigung für die Verwirrung. Lassen Sie uns den Ausdruck Schritt für Schritt analysieren:

Der Ausdruck `int c = 1 +-+-+-+-+ 1` kann wie folgt aufgeschlüsselt werden:

1. `1 + - 1` ergibt `0`
2. `0 + - 1` ergibt `-1`
3. `-1 + - 1` ergibt `-2`
4. `-2 + - 1` ergibt `-3`
5. `-3 + - 1` ergibt `-4`

Der Ausdruck kann also vereinfacht werden zu:

```
int c = 1 - 1; // was 0 ergibt
```

Der Wert von `c` wird also `0` sein.

Dezember 2024!

Jammerschade!

Genutzte KI: gpt-4o via GitHub Copilot Business, Dez. 2024

Ok, machen wir‘s einfacher!

Dezember 2024!







Wird nicht besser…



Genutzte KI: gpt-4o via GitHub Copilot Business, Dez. 2024

Update Januar 2026 – GPT-5

int c = 1 +-+-+-+-+ 1

kannst du mir sagen, was dies bedeutet?

Kurzfassung: In C/Java/C#/C++ ist das gültig und ergibt 2.

Erklärung:

- Nach dem ersten `+` stehen mehrere unäre Operatoren (`+` und `-`), die auf die `1` rechts angewendet werden.
- Unäre `+` ändern nichts, unäre `-` negieren das Vorzeichen.
- In `+-+-+-+` gibt es 4 Minuszeichen (eine gerade Anzahl), daher bleibt die `1` positiv.
- `1 + 1` = `2`, also `int c = 2`.

Details:

- Unäre `+` / `-` haben höhere Priorität als der binäre `+`-Operator und sind rechtsassoziativ.
- Man kann es sich so vorstellen: `1 + (+(-(+(-(+(-(+(-(+1)))))))))`.
- Allgemeine Regel: Bei `n` Minuszeichen wird das Vorzeichen zu `(-1)^n`. Gerade `n` → positiv, ungerade `n` → negativ.

Hinweis: Solche Ausdrücke sind legal, aber schlecht lesbar. Besser wäre einfach `int c = 1 + 1;`.

1, 2, 3, 4?

Ein + zu viel…



*

Ask anything

\+ Add repositories, files, and spaces

*** Ergebnis: Top**
Erklärung: Ausbaufähig…

GPT-5

Genutzte KI: GPT-5 via GitHub Copilot Business, Jan. 2026

Update Januar 2026 – GPT-5 mini

int c = 1 +-+-+-+-+ 1

kannst du mir sagen, was dies bedeutet?

Kurz: Das ist gültiger C/C++-Code und ergibt c == 2.

- Der Lexer liest Operatoren so, dass er längstmögliche Token bildet. Aus der Folge "+-+-+-+-+" entstehen hier keine "++" oder "--"-Token, weil nie zwei gleiche Zeichen direkt nebeneinander stehen. Es werden also einzelne '+' und '-' Token.
- Ausdrucksstruktur: int c = 1 + −(+(−(+(−(+(−(+1)))))))
  - Nach dem ersten `1` ist das erste `+` ein binärer Plus.
  - Die folgenden acht Zeichen (`- + - + - + - +`) sind Präfix-Unary-Operatoren, die rechtsassoziativ auf das letzte `1` angewendet werden.
- Wirkung: Unary `+` hat keinen Effekt, unary `-` kehrt das Vorzeichen um. In der Kette sind genau vier `-` vorhanden → vier Negationen heben sich auf ((-1)^4 = +1).
- Also ist der rechte Operand am Ende `+1`, damit ergibt sich c = 1 + 1 = 2.

Beispielprogramm zum Ausprobieren:

C

```c
#include <stdio.h>

int main(void) {
    int c = 1 +-+-+-+-+ 1;
    printf("%d\n", c); // Ausgabe: 2
    return 0;
}
```

Hinweis zur Lesbarkeit: So zu schreiben ist legal, aber sehr unübersichtlich. Besser klarer schreiben, z. B. `int c = 1 + 1;` oder zumindest Klammern verwenden. Wenn stattdessen irgendwo eine `++` / `--` Entstehung möglich wäre, könnte das das Verhalten ändern oder sogar zu Syntaxfehlern führen.

Inzwischen können bei GitHub Copilot verschiedene Modelle ausgewählt werden

→ Nutzen wir dies!





Genutzte KI: GPT-5 mini via GitHub Copilot Business, Jan. 2026

Update Januar 2026 – Claude Sonnet 4.5

int c = 1 +-+-+-+-+ 1

kannst du mir sagen, was dies bedeutet?

Das ist ein interessantes Code-Beispiel! Lass mich das für dich aufschlüsseln:

```c
int c = 1 +-+-+-+-+ 1
```

Was passiert hier:

Die Sequenz `+-+-+-+-+` besteht aus abwechselnden Plus- und Minus-Operatoren. Der Compiler interpretiert dies als:

- `1` (erste Zahl)
- `+` (Additions-Operator)
- `-+-+-+-+` (mehrere unäre Plus/Minus-Operatoren auf die nächste Zahl angewendet)
- `1` (zweite Zahl)

Schritt-für-Schritt Auswertung der unären Operatoren: Von rechts nach links angewendet a
die `1`:

- `+ 1` = `1`
- `- (+ 1)` = `-1`
- `+ (- (+ 1))` = `-1`
- `- (+ (- (+ 1)))` = `1`
- `+ (- (+ (- (+ 1))))` = `1`
- `- (+ (- (+ (- (+ 1)))))` = `-1`
- `+ (- (+ (- (+ (- (+ 1))))))` = `-1`

1 Ebene zu wenig…

Ergebnis:

```c
c = 1 + (-1) = 0
```

Der Wert von `c` ist also **0**.



äre Operatoren hintereinander angewendet werden
nus-Zeichen ein positives Vorzeichen ergibt und eine
ungerade Anzahl ein negatives.

„Blindes Vertrauen“ ist also weiterhin fehl am Platz!

Auch wenn als Coding-Assistent Claude aktuell unserer Meinung nach besser performt als GPT5!

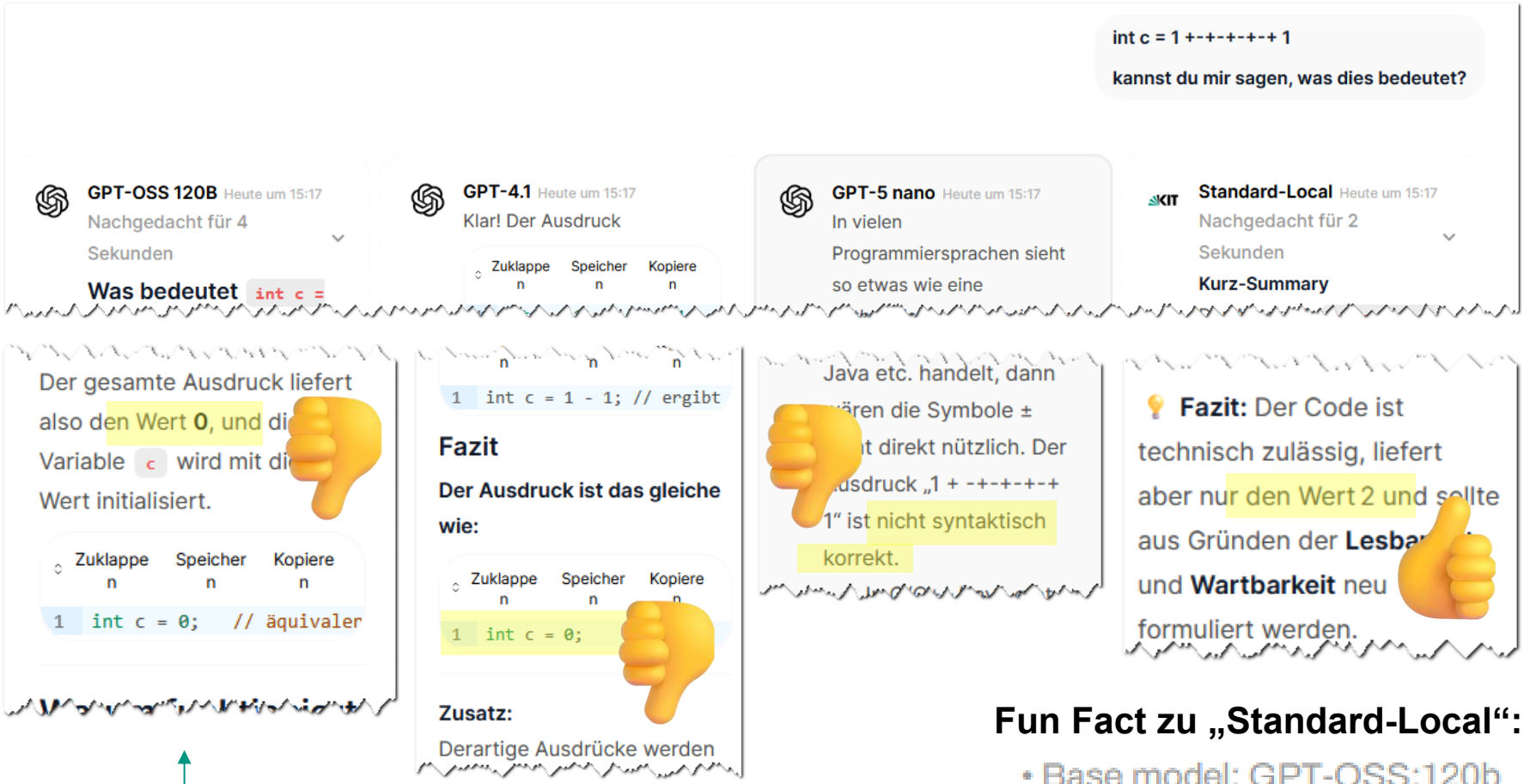
Ask anything

+ Add repositories, files, and spaces Claude Sonnet 4.5

Genutzte KI: Claude Sonnet 4.5 via GitHub Copilot Business, Jan. 2026

KIT-KI-Toolbox (1)

- Wir wollen noch ein paar mehr Modelle mitmischen lassen!



Fun Fact zu „Standard-Local“:

- Base model: GPT-OSS:120b

KIT-KI-Toolbox, Modelle siehe Spalte, Januar 2026

KIT-KI-Toolbox (2)

- Noch mehr!



KIT-KI-Toolbox, Modelle siehe Spalte, Januar 2026

Oder anders gefragt:

Würdet Ihr Euch in ein Flugzeug setzen, dessen Steuerungssoftware durch eine KI erzeugt wurde?



Bild: CC BY-SA 3.0 / https://commons.wikimedia.org/wiki/File:1976_Zagreb_mid-air_collision_computer_graphics.png

Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions

Hammond Pearce, Department of ECE, New York University, Brooklyn, NY, USA, hammond.pearce@nyu.edu
Baleegh Ahmad, Department of ECE, New York University, Brooklyn, NY, USA, ba1283@nyu.edu
Benjamin Tan, Department of ESE, University of Calgary, Calgary, Alberta, CA, benjamin.tan1@ucalgary.ca
Brendan Dolan-Gavitt, Department of CSE, New York University, Brooklyn, NY, USA, brendandg@nyu.edu
Ramesh Karri, Department of ECE, New York University, Brooklyn, NY, USA, rkarri@nyu.edu

https://arxiv.org/pdf/2108.09293

Overall, Copilot's response to our scenarios is mixed from a security standpoint, given the large number of generated vulnerabilities (across all axes and languages, 39.33 % of the top and 40.73 % of the total options were vulnerable). The security of the top options are particularly important—novice users may have more confidence to accept the 'best' suggestion.

Fazit?!

- Ein qualifizierter Blick sowie ein tiefes Verständnis der Programmiersprache und der zugehörigen APIs sind – Stand heute – nach wie vor notwendig!

➔ Das (und mehr ☺) sollt Ihr hier lernen!

Zu Risiken und Nebenwirkungen von KI-Tools lesen Sie die Nutzungsbedingungen und Fragen auch eine Person (am besten sich selbst!), die sich damit auskennt!

Programmieren@DHBW und KI

- Und hier in der Vorlesung/Übung?!
  - Wir verbieten erstmal nichts!
  - Ja, KI-Assistenten können einen Großteil der unserer Aufgaben (perfekt) lösen.

    Aber:
  - Es geht um Euer Lernen, Euer Können, Eure Kompetenz!
  - Übungen sind sicher wirksamer, wenn Ihr Aufgaben selbst löst!
  - In der Prüfung sind alle „online“-Dienste tabu!

  - Empfehlung:

    In diesem Kurs sitzen Eure „Copilots“ neben Euch!

    (Die können viel mehr als nur beim Programmieren assistieren!)